Route the URL via Path Configuration

Before we can navigate to the new fragment, `MapFragment`, we have to make Hotwire Native aware of it. In `HikingJournalApplication.kt`, register this and the default `HotwireWebFragment` with the framework.

ch06_14/android/app/src/main/java/com/masilotti/hikingjournal/HikingJournalApplication.kt

```
  package com.masilotti.hikingjournal

  import android.app.Application
  import com.masilotti.hikingjournal.activities.baseURL
» import com.masilotti.hikingjournal.fragments.MapFragment
  import dev.hotwire.core.config.Hotwire
» import dev.hotwire.core.turbo.config.PathConfiguration
» import dev.hotwire.navigation.config.registerFragmentDestinations
» import dev.hotwire.navigation.fragments.HotwireWebFragment

  class HikingJournalApplication : Application() {
      override fun onCreate() {
          super.onCreate()

          Hotwire.loadPathConfiguration(
              context = this,
              location = PathConfiguration.Location(
                  remoteFileUrl = "$baseURL/configurations/android_v1.json"
              )
          )

»         Hotwire.registerFragmentDestinations(
»             HotwireWebFragment::class,
»             MapFragment::class,
»         )
      }
  }
```

Then, open `MapFragment.kt` and decorate the class with `@HotwireDestinationDeepLink`. We'll use this URI in the path configuration to